

The CDC-P as a Software ASIC

Reconfigurable Flip-Flop Cascades and Dynamic Clock Acceleration in a Cooperative Deterministic Kernel

Marcos Roberto Braga
Informal Institute of Education, Science and Technology
noelmr_b_tec@yahoo.com

Pedro Henrique Cerqueira Braga
Collaborator

September 12, 2026

Abstract

This work proposes a conceptual equivalence between the CDC-P (Concurrency Deterministic Control with Preemption) kernel and a synchronous ASIC (Application-Specific Integrated Circuit). We argue that CDC-P tasks behave as flip-flops, the tick behaves as a clock, and the dispatcher behaves as combinational logic. However, unlike a physical ASIC, CDC-P exhibits five capabilities that an ASIC lacks: (i) runtime reconfiguration of the flip-flop cascade (queue reordering via URG-S); (ii) dynamic clock acceleration under load (tick adjustment and turbo mode); (iii) per-task temporal self-regulation (homeostasis); (iv) architectural fault isolation (Sheriff, Ombudsman, permanent pane); and (v) radical portability (same kernel across five architectures). We call this model a *Software ASIC*: it preserves the determinism of synchronous hardware while offering the flexibility of software. Empirical results on PIC16, PIC18, AVR, RISC-V, and ARM Cortex-M demonstrate the feasibility of the model.

Keywords: ASIC, CDC-P, synchronous hardware, reconfigurable logic, dynamic clock, deterministic kernel, embedded systems.

1 Introduction

In synchronous digital design, an ASIC (Application-Specific Integrated Circuit) is a hardware device in which a fixed cascade of flip-flops is synchronized by a single clock. The behavior of the circuit is deterministic: at each clock edge, the flip-flops capture their next state according to combinational logic. This determinism is a physical property of silicon.

In this work, we propose that the CDC-P kernel exhibits the same deterministic behavior as a synchronous ASIC, but with a crucial difference: **the CDC-P can reconfigure its own flip-flop cascade and accelerate its own clock at runtime**. This capability does not exist in a physical ASIC.

We argue that the CDC-P is, in fact, a *Software ASIC*: a kernel that behaves like synchronous hardware while retaining the flexibility of software.

1.1 Contributions

- A formal equivalence between CDC-P tasks and flip-flops;
- A formal equivalence between the tick and a global clock;
- An analysis of five capabilities that CDC-P has and ASICs lack;
- A design framework for building software-reconfigurable deterministic systems;
- Empirical validation on five architectures.

2 The ASIC Model

2.1 Synchronous Hardware

An ASIC is characterized by:

- A set of **flip-flops** that store state;
- A **global clock** that synchronizes all flip-flops;
- **Combinational logic** that computes the next state;
- A **fixed datapath** defined at fabrication time;
- **Deterministic behavior** guaranteed by physics.

At each clock edge, every flip-flop captures the value produced by the combinational logic. The result is deterministic, reproducible, and independent of external events (except for inputs).

2.2 Properties of a Synchronous ASIC

Table 1: Properties of a synchronous ASIC

Property	Description
Determinism	Guaranteed by physics
Clock	Fixed by oscillator
Datapath	Fixed at fabrication
Reconfiguration	Impossible
Fault isolation	Not supported
Portability	Application-specific

3 The CDC-P as a Software ASIC

3.1 Structural Equivalence

Table 2: Structural equivalence between ASIC and CDC-P

ASIC	CDC-P
Flip-flops	Tasks (functions that return)
Global clock	Tick (timer or SysTick)
Combinational logic	Dispatcher
Datapath	Priority queue
Fabrication	Kernel configuration

At each tick, every task is evaluated by the dispatcher according to modular arithmetic:

$$\text{execute task } i \iff (tick - timer_i) \geq period_i \quad (1)$$

This is mathematically deterministic, just as a flip-flop's next state is deterministic in hardware.

3.2 What the CDC-P Has That an ASIC Does Not

3.2.1 Runtime Reconfiguration of the Flip-Flop Cascade

In an ASIC, the cascade of flip-flops is fixed at fabrication. In CDC-P, the queue can be reordered at runtime through URG-S (Situational Urgency):

```
set_urg_task2(); // Reorders execution at next dispatcher scan
```

This allows a task to *jump the queue* without interrupting other tasks. The effect is equivalent to reconfiguring the datapath of an ASIC on the fly.

3.2.2 Dynamic Clock Acceleration

In an ASIC, the clock is fixed by an oscillator. In CDC-P, the tick can be adjusted at runtime:

Table 3: Dynamic tick modes in CDC-P

Mode	Tick	Use
Turbo	1 ms	Critical control
Fast	2 ms	Intensive monitoring
Normal	8 ms	General purpose
Slow	50 ms	Low-power logging

The kernel can switch modes automatically based on load (Task8 — Accelerator) or on application request (command 'd').

3.2.3 Per-Task Temporal Self-Regulation

In an ASIC, no element can slow itself down. In CDC-P, each task self-regulates:

```
timer_ex_task1 = (tick - timer_task1) / task1_periodo;  
if(timer_ex_task1) { task1_periodo = task1_periodo + 1; }
```

This is homeostasis: the task slows down under load, preventing system collapse.

3.2.4 Architectural Fault Isolation

An ASIC cannot isolate a faulty flip-flop. CDC-P isolates faulty tasks through:

- **Sheriff (Task10):** detects chronic failure and isolates the task permanently;
- **Ombudsman (Task9):** recovers blocked tasks;
- **Fail-safe:** resets the system if the Ombudsman fails;
- **Permanent pane:** blocks a task until external intervention.

This is equivalent to reconfiguring the ASIC to bypass a faulty region.

3.2.5 Radical Portability

An ASIC is application-specific. CDC-P is portable across:

- PIC16, PIC18 (8-bit);
- AVR AT90S8515 (8-bit);
- RISC-V CH32V003 (32-bit);
- ARM Cortex-M3 STM32F103 (32-bit).

The same kernel code runs on all platforms.

4 Empirical Validation

4.1 Platforms

Table 4: Tested platforms

MCU	Architecture	Clock	RAM	Flash
PIC16F628A	8 bits	4 MHz	224 B	2 KB
PIC18F6722	8 bits	8 MHz	4 KB	128 KB
AT90S8515	AVR 8 bits	8 MHz	512 B	8 KB
CH32V003F4P6	RISC-V	48 MHz	2 KB	16 KB
STM32F103	ARM Cortex-M3	72 MHz	20 KB	64 KB

4.2 Determinism

In all platforms, the second counter remained exact under load. No freezing, no jumping. This confirms the equivalence with synchronous hardware.

4.3 Reconfiguration

URG-S was tested at 1000 events/s. The system remained stable, and urgent tasks executed on the next dispatcher scan, with latency of 25–112 μ s.

4.4 Dynamic Clock

The tick was switched between 1, 2, 8, and 50 ms at runtime. No glitches were observed; the system adapted smoothly.

4.5 Fault Isolation

The Sheriff and Ombudsman mechanisms isolated deliberately faulty tasks without affecting healthy ones.

5 Discussion

5.1 Why This Matters

ASICs are fast and deterministic but inflexible. Software is flexible but often non-deterministic. CDC-P offers the best of both worlds:

- Determinism of synchronous hardware;
- Flexibility of software;
- Reconfiguration at runtime;
- Portability across architectures.

5.2 Where CDC-P Excels

- Hard real-time systems with periodic tasks;
- Embedded systems with resource constraints;
- Systems requiring fault tolerance;
- Systems requiring dynamic adaptation.

5.3 Where ASICs Are Superior

- Extreme performance (GHz);
- Extreme energy efficiency (pJ/op);
- Massive parallelism (thousands of parallel paths);
- Production volumes (millions of units).

The two approaches are complementary, not competing.

6 Conclusion

We have proposed a conceptual equivalence between CDC-P and a synchronous ASIC, and identified five capabilities that CDC-P has and ASICs lack: (i) runtime reconfiguration of the flip-flop cascade; (ii) dynamic clock acceleration; (iii) per-task temporal self-regulation; (iv) architectural fault isolation; and (v) radical portability.

We call this model a *Software ASIC*: a kernel that behaves like synchronous hardware while retaining the flexibility of software.

“CDC-P is an ASIC that can rewrite its own silicon at runtime.”

Acknowledgments

The author thanks Pedro Henrique Cerqueira Braga for his collaboration in the development and testing of CDC-P.

References

- [1] LIU, C. L.; LAYLAND, J. W. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, v. 20, n. 1, p. 46–61, 1973.
- [2] DIJKSTRA, E. W. Go To Statement Considered Harmful. *Communications of the ACM*, v. 11, n. 3, p. 147–148, 1968.
- [3] DIJKSTRA, E. W. The Humble Programmer. *Communications of the ACM*, v. 15, n. 10, p. 859–866, 1972.

- [4] WESTE, N.; HARRIS, D. *CMOS VLSI Design: A Circuits and Systems Perspective*. 4th ed. Addison-Wesley, 2010.
- [5] RABAEY, J.; CHANDRAKASAN, A.; NIKOLIC, B. *Digital Integrated Circuits: A Design Perspective*. 2nd ed. Prentice Hall, 2003.
- [6] FreeRTOS Documentation. <https://www.freertos.org/>. Accessed: 2026.
- [7] BARR, M. *Programming Embedded Systems in C and C++*. O'Reilly, 1999.
- [8] LABROSSE, J. J. *$\mu C/OS-III$: The Real-Time Kernel*. Micrium, 2010.
- [9] JONES, M. What Really Happened on Mars? *NASA JPL*, 1997.
- [10] ASTIER, J. Y. Multiple FSMs vs RTOS. *LinkedIn*, 2024.